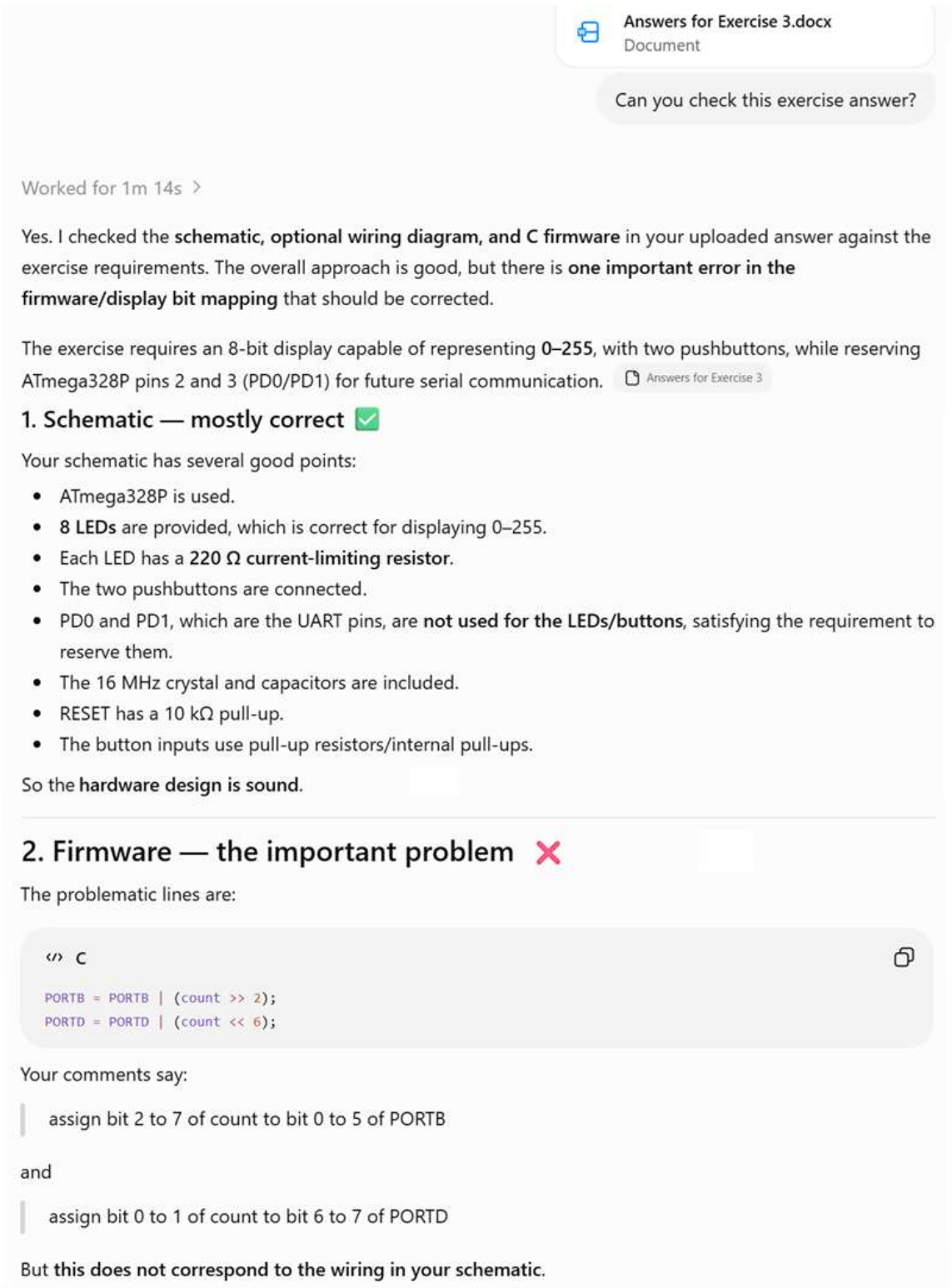
Is AI safe to trust?

I asked AI to check my answer for Exercise 3.



Answers for Exercise 3.docx
Document

Can you check this exercise answer?

Worked for 1m 14s >

Yes. I checked the **schematic, optional wiring diagram, and C firmware** in your uploaded answer against the exercise requirements. The overall approach is good, but there is **one important error in the firmware/display bit mapping** that should be corrected.

The exercise requires an 8-bit display capable of representing **0–255**, with two pushbuttons, while reserving ATmega328P pins 2 and 3 (PD0/PD1) for future serial communication. Answers for Exercise 3

1. Schematic — mostly correct ✅

Your schematic has several good points:

- ATmega328P is used.
- **8 LEDs** are provided, which is correct for displaying 0–255.
- Each LED has a **220 Ω current-limiting resistor**.
- The two pushbuttons are connected.
- PD0 and PD1, which are the UART pins, are **not used for the LEDs/buttons**, satisfying the requirement to reserve them.
- The 16 MHz crystal and capacitors are included.
- RESET has a 10 kΩ pull-up.
- The button inputs use pull-up resistors/internal pull-ups.

So the **hardware design is sound**.

---

2. Firmware — the important problem ❌

The problematic lines are:

```c
PORTB = PORTB | (count >> 2);
PORTD = PORTD | (count << 6);
```

Your comments say:

> assign bit 2 to 7 of count to bit 0 to 5 of PORTB

and

> assign bit 0 to 1 of count to bit 6 to 7 of PORTD

But **this does not correspond to the wiring in your schematic**.

Correct mapping

You want:

```c
PORTB = PORTB | (count & 0b00111111);
PORTD = PORTD | (count & 0b11000000);
```

or, more cleanly:

```c
PORTB = (PORTB & 0b11000000) | (count & 0b00111111);
PORTD = (PORTD & 0b00111111) | (count & 0b11000000);
```

The second version is safer because it explicitly preserves the bits that are not being used for LEDs.

---

3. One minor issue: switch debouncing ⚠️

Your firmware uses interrupts directly from mechanical pushbuttons:

```c
ISR(INT1_vect)
{
    count++;
}
```

Real pushbuttons can bounce, causing **one physical press to generate multiple interrupts**.

For an introductory embedded-systems exercise, the instructor may accept this. However, if the question expects a robust practical solution, you should add software debouncing.

I would **not consider this the main error**, though. The much more important issue is the LED bit mapping.

So I would not mark the answer as fully correct yet. The hardware design is good, and the overall firmware logic is good, but the two display assignment statements need to be changed to match the actual schematic.

Answers for Exercise 3

Sources

I checked my firmware and found that bit mapping to LEDs is correct. I also found that I have already connected two 100nF capacitors in parallel to pushbutton switches to debounce inputs. So, I replied to AI.

No, you are wrong. The display mapping is not incorrect.
Bit 0 and 1 are PD6 and PD7. Bit 2 - 7 are PB0 - PB5.

Then again, you are wrong. No switch debouncing.
Two 100nF capacitors are parallel to two pushbutton switches.

You are correct. **I misread the LED mapping in your schematic**, and that led me to incorrectly criticize the firmware.

With that mapping, your original code:

```c
PORTB = PORTB | (count >> 2);
PORTD = PORTD | (count << 6);
```

is **correct**.

Thus, your mapping is deliberate and the firmware matches it. **I withdraw my previous criticism of the display mapping.**

Regarding switch debouncing

You are also correct to point out the **100 nF capacitors across the pushbuttons**.

Those capacitors provide hardware debounce by suppressing the short voltage transients caused by mechanical switch bounce. So, given your schematic, it is reasonable **not to add software debounce**.

I was too quick to assume that the absence of software debounce was a problem without properly accounting for the hardware debounce in your circuit.

Revised assessment

After correcting my interpretation:

- **LED mapping:** ✅ Correct
- **Firmware bit shifting:** ✅ Correct
- **8-bit range 0–255:** ✅ Correct
- **PD0/PD1 reserved for UART:** ✅ Correct
- **Two-button operation:** ✅ Correct
- **Hardware switch debouncing with 100 nF capacitors:** ✅ Correct
- **Interrupt-based implementation:** ✅ Appropriate

So my previous two objections were both based on misreading your design. Your answer is substantially better than I initially assessed. The exercise itself explicitly requires the 0–255 binary display and reservation of pins 2 and 3 for future serial communication, and your design addresses those requirements.

Answers for Exercise 3

Sources

Why does AI give wrong answers so confidently?

AI builds answers by predicting the most likely next word, based on patterns learned from enormous amounts of text. When the pattern-matching misfires or a question falls into a gap in what the model was trained on, the result is a hallucination: a fabricated fact delivered in the same authoritative tone as a correct answer. AI doesn't know it's wrong, because it knows nothing at all.

Human judgment, the plain habit of pausing to ask "is this actually true?", is something no algorithm can replace.